

Building AI Agents with LangGraph

 Demo 1

Demo: Building a Finance Bot with LangGraph

Overview

FinAdvise is an AI-powered personal finance assistant built as a Streamlit web application. It simplifies financial management by enabling users to track expenses, access real-time stock information via the Alpha Vantage API, view budget summaries, and receive personalized financial advice through a conversational interface. Powered by LangGraph and the Groq API, FinAdvise maintains user context across sessions, provides empathetic and clear responses, and incorporates a Human-in-the-Loop (HITL) mechanism for high-risk queries. Recent updates include improved stock symbol extraction, robust error handling for Alpha Vantage API calls, and logging for debugging.

Scenario

Imagine a 30-year-old user, Sarah, who earns \$50,000 annually and wants to save for a car while managing daily expenses. Sarah struggles with fragmented financial tools and limited financial literacy. She interacts with FinAdvise, asking:

- "What's the price of Apple stock?"
- "Add \$50 for groceries."
- "What's a good budget plan for me?"
- "Should I liquidate my retirement account to buy a house?"

FinAdvise collects Sarah's profile (age, income, goals), tracks expenses, provides real stock data, suggests budgets, and flags high-risk queries for expert review, using clear, empathetic language.

Problem Statement

Individuals face challenges in personal finance due to:

- **Fragmented Tools:** Disjointed apps for budgeting, investing, and expense tracking.
- **Overwhelming Data:** Conflicting online financial content confuses users.
- **Generic Advice:** One-size-fits-all recommendations ignore unique goals or risk tolerance.
- **Lack of Memory:** Chatbots forget context, leading to repetitive interactions.
- **Financial Literacy Gap:** Users need clear, empathetic explanations.

Approach for the Solution

The development approach includes:

- **User Data Collection:** Gather profile details through conversation, stored in state and Streamlit session state.
- **State Schema:** Track user input, intent, profile, memory, and HITL flags.
- **Modular Nodes:** LangGraph nodes for:
 - Intent detection.
 - User data collection.
 - Stock info (Alpha Vantage), expense tracking, budget summaries, advice.
 - HITL and fallback.
- **Memory Management:** Short-term (in-session) and long-term (cross-session) memory.
- **HITL Integration:** Flag high-risk queries (e.g., "liquidate") for review.
- **Stock Data Integration:** Use Alpha Vantage API with regex-validated symbol extraction and detailed error handling.
- **Debugging:** Log API responses and errors for troubleshooting.

Stepwise Solution

FinAdvise offers an integrated AI assistant with:

- Unified Interface: Combines stock queries (Alpha Vantage), expense tracking, budget summaries, and advice in a chat-based platform.
- Contextual Memory: Short-term (in-session) and long-term (cross-session) memory for personalized conversations.
- Personalized Advice: Collects user data (age, income, goals, risk tolerance) for tailored responses.
- Empathetic Responses: Clear, jargon-free language for users with limited financial literacy.
- HITL Mechanism: Flags high-risk queries for simulated human review.
- Robust Stock Data: Real-time stock prices via Alpha Vantage with improved symbol extraction and error handling.
- Debugging: Logging for API issues.

Project Structure

finadvise/



```
├── env4/          # Virtual environment directory
├── .env           # Environment variables (GROQ_API_KEY, ALPHA_VANTAGE_API_KEY)
├── app.py         # Main Streamlit application
├── requirements.txt # Python dependencies
├── README.md      # Basic project documentation
├── FinAdvise_Documentation.md # Comprehensive documentation
└── .gitignore     # Excludes env4/, .env
```

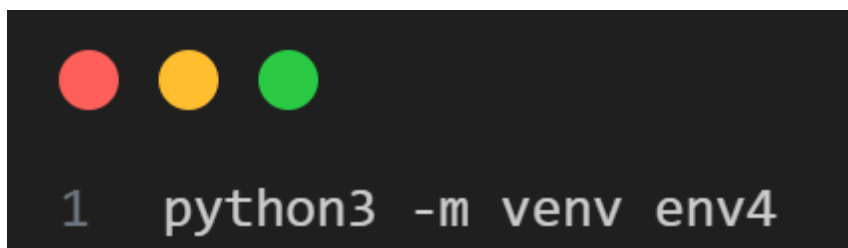
- **.env**: Stores Groq and Alpha Vantage API keys.
- **app.py**: Core application with Streamlit UI, LangGraph, LLM, and Alpha Vantage integration.
- **requirements.txt**: Lists dependencies (streamlit, langchain, requests, etc.).
- **README.md**: Quick setup and usage guide.
- **.gitignore**: Prevents sensitive data exposure.

Step 1: Setting up the Environment

- **Objective:** Follow these steps to set up and run FinAdvise in the env4 virtual environment.
- **Prerequisites**
 - Python 3.8 or higher
 - Groq API key (Groq)
 - Alpha Vantage API key (Alpha Vantage)

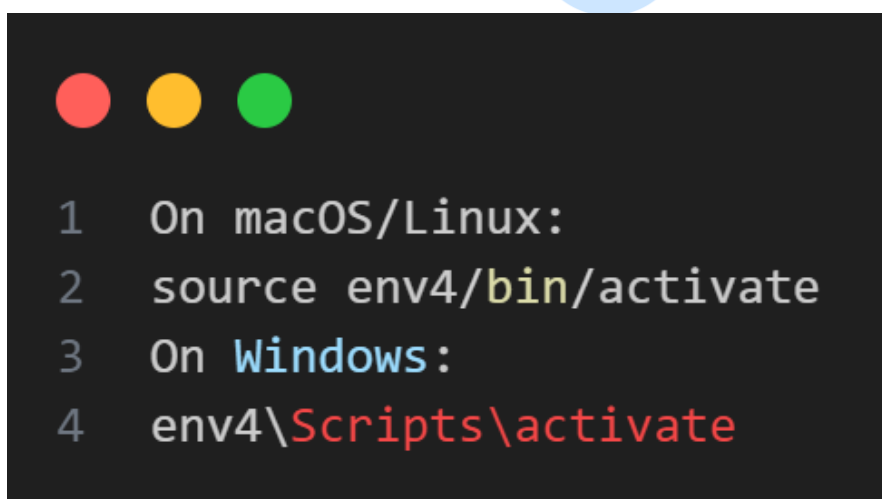
Step 2: Create Virtual Environment

- **Objective:** Create a virtual environment named env4:



```
1 python3 -m venv env4
```

Step 3: Activate Virtual Environment



```
1 On macOS/Linux:  
2 source env4/bin/activate  
3 On Windows:  
4 env4\Scripts\activate
```

Step 4: Install Dependencies

Install required packages from requirements.txt:

```
pip install -r requirements.txt
```

Contents of requirements.txt:



```
1 streamlit>=1.30.0
2 langchain>=0.1.14
3 langchain_groq>=0.1.4
4 langgraph>=0.0.35
5 python-dotenv>=1.0.0
6 requests>=2.31.0
```

Step 5: Configure Environment Variables

Create a .env file in the project root with your API keys:

- GROQ_API_KEY=<your-groq-api-key>
- ALPHA_VANTAGE_API_KEY=<your-alpha-vantage-api-key>

Step 6: Run the Application

Start the Streamlit app:

```
streamlit run app.py
```

Open <http://localhost:8501> in your browser.

Step 7: Deactivate Virtual Environment

When done, deactivate the environment:

```
deactivate
```

NOTES:

- Ensure .gitignore includes env4/ and .env.
- Verify Python version (python --version).

- Upgrade pip if needed: `python -m pip install --upgrade pip`.
- Alpha Vantage free tier limits to 5 API calls/minute.



Step-by-Step Code Explanation

Step 1: Import necessary libraries

```
1 import streamlit as st
2 import asyncio
3 from langgraph.graph import StateGraph
4 from langchain_groq import ChatGroq
5 from typing import TypedDict, Optional, Dict
6 import re
7 from dotenv import load_dotenv
8 import os
9 import requests
10 import logging
```

- `streamlit as st`: Imports the Streamlit library, which is used to build the web interface. `st` is the conventional alias for Streamlit.
- `asyncio`: Imports the asyncio library, which is used for writing concurrent code using the `async/await` syntax. This is necessary because the LLM and API calls are asynchronous.
- `from langgraph.graph import StateGraph`: Imports the `StateGraph` class from the LangGraph library. LangGraph helps build conversational agents with structured workflows.
- `from langchain_groq import ChatGroq`: Imports the `ChatGroq` class from the `langchain_groq` integration. This class allows you to interact with Groq's large language models.
- `from typing import TypedDict, Optional, Dict`: Imports type hinting utilities from the typing module. `TypedDict` is used to create dictionary types with specific keys and value types, `Optional` indicates that a variable can be `None`, and `Dict` is for standard dictionaries.
- `import re`: Imports the regular expression module (`re`), used for pattern matching in strings (e.g., extracting stock symbols).

- `from dotenv import load_dotenv`: Imports the `load_dotenv` function from the `python-dotenv` library. This is used to load environment variables from a `.env` file.
- `import os`: Imports the `os` module, which provides a way of interacting with the operating system (e.g., accessing environment variables).
- `import requests`: Imports the `requests` library, used for making HTTP requests (e.g., to the Alpha Vantage API).
- `import logging`: Imports the `logging` module, used for recording events that occur while the program runs.

Step 2: Configuration



```
1  # === CONFIG ===
2  load_dotenv()
3  GROQ_API_KEY = os.getenv("GROQ_API_KEY")
4  ALPHA_VANTAGE_API_KEY = os.getenv("ALPHA_VANTAGE_API_KEY")
5
6  # Setup logging
7  logging.basicConfig(level=logging.INFO)
8  logger = logging.getLogger(__name__)
```

- `# === CONFIG ===`: This is a comment indicating a section for configuration.
- `load_dotenv()`: Calls the function to load environment variables from a `.env` file in the same directory as the script. This is where API keys and other sensitive information should be stored.
- `GROQ_API_KEY = os.getenv("GROQ_API_KEY")`: Retrieves the Groq API key from the environment variables. It assumes you have a variable named `GROQ_API_KEY` in your `.env` file.
- `ALPHA_VANTAGE_API_KEY = os.getenv("ALPHA_VANTAGE_API_KEY")`: Retrieves the Alpha Vantage API key from the environment variables. It assumes you have a variable named `ALPHA_VANTAGE_API_KEY` in your `.env` file.

- `logging.basicConfig(level=logging.INFO)`: Configures the basic logging setup. It sets the logging level to INFO, meaning that informational messages and more severe messages will be logged.
- `logger = logging.getLogger(__name__)`: Creates a logger object associated with the current module (`__name__`). This allows you to log messages with context.

Step 3: Constants

```
1  # === CONSTANTS ===  
2  INTENT_DETECTION_NODE = "Intent Detection"
```

- `# === CONSTANTS ===`: This comment indicates a section for defining constants.
- `INTENT_DETECTION_NODE = "Intent Detection"`: Defines a constant string that will be used as the name of the intent detection node in the LangGraph. This improves readability and makes it easier to refer to this node.

Step 4: State Definition

```
1  # === STATE ===  
2  class FinanceState(TypedDict):  
3      user_input: str  
4      intent: Optional[str]  
5      data: Optional[dict]  
6      user_profile: Optional[Dict[str, str]] # Age, income, goals, risk tolerance  
7      short_term_memory: Optional[Dict[str, str]] # In-session memory  
8      long_term_memory: Optional[Dict[str, str]] # Cross-session memory  
9      hitl_flag: Optional[bool] # Flag for high-risk queries
```

- `# === STATE ===`: This comment indicates the definition of the application's state.
- `class FinanceState(TypedDict):`: Defines a typed dictionary called `FinanceState`. This class specifies the structure of the data that will be passed between the nodes in the LangGraph.
- `user_input: str`: The latest input from the user.

- **intent:** Optional[str]: The detected intent of the user's input (e.g., "stock", "advice"). It's Optional because it might not be determined yet.
- **data:** Optional[dict]: A dictionary to store the output or response from a specific node (e.g., stock information, advice).
- **user_profile:** Optional[Dict[str, str]]: A dictionary to store information about the user, such as age, income, financial goals, and risk tolerance. It's Optional and defaults to an empty dictionary.
- **short_term_memory:** Optional[Dict[str, str]]: A dictionary to store information relevant to the current session, like the previous intent or the last asked question. It's Optional and defaults to an empty dictionary.
- **long_term_memory:** Optional[Dict[str, str]]: A dictionary to store information that persists across sessions, like the last piece of advice given. It's Optional and defaults to an empty dictionary.
- **hitl_flag:** Optional[bool]: A boolean flag to indicate if the user's query is considered high-risk and requires human intervention. It's Optional and defaults to False.

Step 5: Language Model Initialization



```
1 # === LLM ===  
2 llm = ChatGroq(groq_api_key=GROQ_API_KEY, model_name="llama3-70b-8192")
```

- **# === LLM ===:** This comment indicates the initialization of the language model.
- **llm = ChatGroq(groq_api_key=GROQ_API_KEY, model_name="llama3-70b-8192"):** Creates an instance of the ChatGroq class.
 - **groq_api_key=GROQ_API_KEY:** Passes the Groq API key obtained from the environment variables.
 - **model_name="llama3-70b-8192":** Specifies the Groq model to be used (in this case, "llama3-70b-8192").

Step 6: User Profile Collection Function

```

1 # === USER PROFILE COLLECTION ===
2 async def collect_user_data(state: FinanceState) -> FinanceState:
3     user_input = state['user_input']
4     user_profile = state.get('user_profile', {})
5     short_term_memory = state.get('short_term_memory', {})
6
7     prompt = (
8         f"Extract user profile information (age, income, financial goals, risk tolerance) from: {user_input}. "
9         f"Current profile: {user_profile}. "
10        f"If no new information is provided, ask a question to gather missing data (e.g., 'How old are you?' or 'What are your financial goals?'). "
11        f"Keep tone empathetic and clear."
12    )
13    response = await llm.ainvoke(prompt)
14    message = response.content.strip()
15
16    if "age:" in message.lower() or "income:" in message.lower() or "goal:" in message.lower() or "risk:" in message.lower():
17        for line in message.split('\n'):
18            if ':' in line:
19                key, value = line.split(':', 1)
20                user_profile[key.lower()] = value
21    else:
22        short_term_memory['last_question'] = message
23
24    return {**state, "user_profile": user_profile, "data": {"response": message}, "short_term_memory": short_term_memory}

```

- `# === USER PROFILE COLLECTION ===`: Comment indicating the user profile collection logic.
- `async def collect_user_data(state: FinanceState) -> FinanceState`: Defines an asynchronous function that takes the current `FinanceState` and returns an updated `FinanceState`.
- It extracts `user_input`, `user_profile`, and `short_term_memory` from the state.
- It constructs a prompt for the LLM to extract user profile information from the `user_input`, considering the `current_profile`. If no new information is found, it instructs the LLM to ask a clarifying question.
- `response = await llm.ainvoke(prompt)`: Asynchronously calls the Groq LLM with the constructed prompt.
- `message = response.content.strip()`: Extracts the text content from the LLM's response and removes leading/trailing whitespace.
- It checks if the LLM's response contains keywords indicating user profile information (age, income, goal, risk). If so, it parses the response and updates the `user_profile` dictionary.
- If the LLM's response doesn't contain profile information, it assumes the response is a question and stores it in the `short_term_memory` under the key 'last_question'.
- Finally, it returns a new `FinanceState` dictionary, updating the `user_profile`, setting the `data` key with the LLM's message, and updating the `short_term_memory`.

Step 7: Intent Detection Function

```

1  # === INTENT DETECTION ===
2  async def detect_intent(state: FinanceState) -> FinanceState:
3      user_input = state['user_input']
4      short_term_memory = state.get('short_term_memory', {})
5      long_term_memory = state.get('long_term_memory', {})
6
7      prompt = (
8          f"Classify the user's intent into one of: 'profile', 'stock', 'expense', 'budget', 'advice', or 'unknown'.\n"
9          f"User input: {user_input}\n"
10         f"Previous intent: {short_term_memory.get('previous_intent', 'none')}\n"
11         f"Long-term context: {long_term_memory.get('last_advice', 'none')}\n"
12         f"Intent:"
13     )
14     response = await llm.ainvoke(prompt)
15     content = response.content.strip().lower()
16
17     match = re.search(r"(profile|stock|expense|budget|advice)", content)
18     intent = match.group(1) if match else "unknown"
19     short_term_memory['previous_intent'] = intent
20
21     high_risk_keywords = ["liquidate", "retirement", "all my savings", "entire portfolio"]
22     hitl_flag = any(keyword in user_input.lower() for keyword in high_risk_keywords)
23
24     return {**state, "intent": intent, "short_term_memory": short_term_memory, "hitl_flag": hitl_flag}

```

- # === INTENT DETECTION ===: Comment indicating the intent detection logic.
- `async def detect_intent(state: FinanceState) -> FinanceState::` Defines an asynchronous function that takes the current `FinanceState` and returns an updated `FinanceState` with the detected intent.
- It extracts `user_input`, `short_term_memory`, and `long_term_memory` from the state.
- It constructs a prompt for the LLM to classify the user's intent based on the `user_input`, considering the `previous_intent` (from short-term memory) and any `last_advice` given (from long-term memory) for context.
- `response = await llm.ainvoke(prompt):` Asynchronously calls the Groq LLM to get the intent.
- `content = response.content.strip().lower():` Extracts and lowercases the LLM's response.
- `match = re.search(r"(profile|stock|expense|budget|advice)", content):` Uses a regular expression to find if the LLM's response contains one of the predefined intents.
- `intent = match.group(1) if match else "unknown":` If a match is found, the extracted intent is assigned; otherwise, the intent is set to "unknown".
- `short_term_memory['previous_intent'] = intent:` Stores the detected intent in the short-term memory for potential contextual use in the next turn.

- It defines a list of high_risk_keywords.
- hitl_flag = any(keyword in user_input.lower() for keyword in high_risk_keywords): Checks if any of the high-risk keywords are present in the user's input (case-insensitive). If so, it sets the hitl_flag to True.
- Finally, it returns a new FinanceState with the detected intent, updated short_term_memory, and the hitl_flag.

Step 8: Stock Information Retrieval Function

```

1  # === STOCK INFO ===
2  async def get_stock_info(state: FinanceState) -> FinanceState:
3      user_input = state['user_input']
4      short_term_memory = state.get('short_term_memory', {})
5      user_profile = state.get('user_profile', {})
6
7      # Extract stock symbol using LLM with strict instructions
8      prompt = (
9          f"Extract the stock symbol (e.g., 'AAPL' for Apple) from the request: {user_input}. "
10         f"Return only the symbol (e.g., 'AAPL') or 'UNKNOWN' if unclear. Do not include extra text."
11     )
12     response = await llm.ainvoke(prompt)
13     stock_symbol = response.content.strip().upper()
14
15     # Validate stock symbol with regex
16     if not re.match(r'^[A-Z]{1,5}$', stock_symbol) or stock_symbol == 'UNKNOWN':
17         message = f"Sorry, I couldn't identify a valid stock symbol from '{user_input}'. Please specify the stock (e.g., 'AAPL' for Apple)."
18         logger.warning(f"Invalid stock symbol extracted: {stock_symbol}")
19     else:
20         # Call Alpha Vantage API
21         url = f"https://www.alphavantage.co/query?function=TIME_SERIES_DAILY&symbol={stock_symbol}&apikey={ALPHA_VANTAGE_API_KEY}"
22         try:
23             response = requests.get(url, timeout=10)
24             response.raise_for_status()
25             data = response.json()
26             logger.info(f"Alpha Vantage API response for {stock_symbol}: {data.keys()}")
27
28             if "Time Series (Daily)" in data:
29                 latest_date = list(data["Time Series (Daily)"].keys())[0]
30                 stock_data = data["Time Series (Daily)"][latest_date]
31                 close_price = stock_data["4. close"]
32                 message = f"The latest closing price for {stock_symbol} is ${close_price} (as of {latest_date})."
33
34             # Add risk tolerance advice
35             risk_tolerance = user_profile.get('risk tolerance', 'unknown')
36             risk_prompt = (
37                 f"Provide a brief note on investing in {stock_symbol} tailored to a user with {risk_tolerance} risk tolerance. "
38                 f"Keep it clear and empathetic."
39             )
40             risk_response = await llm.ainvoke(risk_prompt)
41             message += f"\n{risk_response.content.strip()}"
42         except requests.RequestException as e:
43             message = f"Error from Alpha Vantage: {data['Error Message']}. Please check the stock symbol or try again later."
44             logger.error(f"Alpha Vantage error for {stock_symbol}: {data['Error Message']}")
45             elif "Note" in data and "rate limit" in data["Note"].lower():
46                 message = "Alpha Vantage API rate limit exceeded. Please try again in a minute."
47                 logger.warning(f"Rate limit exceeded for {stock_symbol}: {data['Note']}")
48             else:
49                 message = f"No data available for {stock_symbol}. Please check the symbol or try again later."
50                 logger.error(f"No time series data for {stock_symbol}: {data}")
51         except requests.RequestException as e:
52             message = f"Error fetching data for {stock_symbol}: {str(e)}. Please try again later."
53             logger.error(f"Request error for {stock_symbol}: {str(e)}")
54
55     short_term_memory['last_stock_requested'] = user_input
56     return {"state": state, "data": {"response": message, "short_term_memory": short_term_memory}}
```

- # === STOCK INFO ===: Comment indicating the stock information retrieval logic.

- `async def get_stock_info(state: FinanceState) -> FinanceState::` Defines an asynchronous function to fetch stock data and returns an updated `FinanceState`.
- It extracts `user_input`, `short_term_memory`, and `user_profile` from the state.
- It constructs a prompt for the LLM to specifically extract the stock symbol from the `user_input`. It instructs the LLM to return only the symbol or "UNKNOWN".
- `response = await llm.ainvoke(prompt):` Calls the LLM to get the stock symbol.
- `stock_symbol = response.content.strip().upper():` Cleans and uppercases the extracted symbol.
- It validates the `stock_symbol` using a regular expression (`^[A-Z]{1,5}$` for 1 to 5 uppercase letters) and checks if it's not "UNKNOWN".
 - If the symbol is invalid, it creates an error message and logs a warning.
 - If the symbol is valid, it proceeds to call the Alpha Vantage API.
- It constructs the url for the Alpha Vantage API to get daily time series data for the extracted `stock_symbol`, using the `ALPHA_VANTAGE_API_KEY`.
- It uses a try-except block to handle potential `requests.RequestException` during the API call.
 - `response = requests.get(url, timeout=10):` Makes a GET request to the Alpha Vantage API with a 10-second timeout.
 - `response.raise_for_status():` Raises an exception for bad status codes (e.g., 404, 500).
 - `data = response.json():` Parses the JSON response from the API.
 - It logs the keys of the API response for debugging.
 - It checks if "Time Series (Daily)" is in the data. If so:
 - It extracts the `latest_date` and the corresponding `stock_data`.
 - It retrieves the `close_price`.
 - It constructs a success message with the stock symbol, closing price, and date.
 - It retrieves the user's `risk_tolerance` from the `user_profile`.
 - It creates a `risk_prompt` for the LLM to provide a brief note on investing in the stock tailored to the user's risk tolerance.
 - It calls the LLM to get the `risk_response` and appends it to the message.

- It handles different error conditions from the Alpha Vantage API (e.g., "Error Message", rate limit "Note", no data). It creates appropriate error messages and logs errors or warnings.
- If a `requests.RequestException` occurs, it creates an error message and logs the error.
- `short_term_memory['last_stock_requested'] = user_input`: Stores the user's original stock request in the short-term memory.
- Finally, it returns a new `FinanceState` with the data containing the response message and the updated `short_term_memory`.

Step 9: Mock Expense Tracking Function

```
1  # === MOCK EXPENSE TRACKING ===
2  async def track_expenses(state: FinanceState) -> FinanceState:
3      user_input = state['user_input']
4      short_term_memory = state.get('short_term_memory', {})
5      user_profile = state.get('user_profile', {})
6
7      prompt = (
8          f"Mock adding an expense based on: {user_input}. "
9          f"Consider user profile: {user_profile}. "
10         f"Reply with a confirmation message, e.g., 'Added expense of $50 for groceries.'"
11     )
12     response = await llm.ainvoke(prompt)
13     message = response.content.strip()
14
15     short_term_memory['last_expense'] = user_input
16     return {**state, "data": {"response": message}, "short_term_memory": short_term_memory}
```

- `# === MOCK EXPENSE TRACKING ===`: Comment indicating the mocked expense tracking logic.
- `async def track_expenses(state: FinanceState) -> FinanceState`:: Defines an asynchronous function to simulate tracking expenses and returns an updated `FinanceState`.
- It extracts `user_input`, `short_term_memory`, and `user_profile` from the state.
- It constructs a prompt for the LLM to mock adding an expense based on the `user_input`, taking into account the `user_profile`. It instructs the LLM to reply with a confirmation message.

- `response = await llm.ainvoke(prompt)`: Calls the LLM to generate the mock confirmation.
- `message = response.content.strip()`: Cleans the LLM's response.
- `short_term_memory['last_expense'] = user_input`: Stores the user's expense input in the short-term memory.
- It returns a new `FinanceState` with the data containing the mock confirmation message and the updated `short_term_memory`.

Step 10: Mock Budget Summary Function

```
1 # === MOCK BUDGET SUMMARY ===
2 async def budget_summary(state: FinanceState) -> FinanceState:
3     user_profile = state.get('user_profile', {})
4     prompt = (
5         f"Mock a simple budget summary with categories and totals, tailored to user profile: {user_profile}. "
6         f"Use clear, empathetic language."
7     )
8     response = await llm.ainvoke(prompt)
9     message = response.content.strip()
10    return {**state, "data": {"response": message}}
```

- `# === MOCK BUDGET SUMMARY ===`: Comment indicating the mocked budget summary logic.
- `async def budget_summary(state: FinanceState) -> FinanceState`: Defines an asynchronous function to generate a mock budget summary and returns an updated `FinanceState`.
- It extracts the `user_profile` from the state.
- It constructs a prompt for the LLM to create a mock budget summary with categories and totals, tailored to the `user_profile`, using clear and empathetic language.
- `response = await llm.ainvoke(prompt)`: Calls the LLM to generate the mock budget summary.
- `message = response.content.strip()`: Cleans the LLM's response.
- It returns a new `FinanceState` with the data containing the mock budget summary message.

Step 11: Personalized Advice Function

```

1  # === PERSONALIZED ADVICE ===
2  async def provide_advice(state: FinanceState) -> FinanceState:
3      user_input = state['user_input']
4      user_profile = state.get('user_profile', {})
5      long_term_memory = state.get('long_term_memory', {})
6
7      prompt = (
8          f"Provide personalized financial advice based on: {user_input}. "
9          f"User profile: {user_profile}. "
10         f"Previous advice: {long_term_memory.get('last_advice', 'none')}. "
11         f"Use clear, empathetic language suitable for users with limited financial literacy."
12     )
13     response = await llm.ainvoke(prompt)
14     message = response.content.strip()
15
16     long_term_memory['last_advice'] = message
17     return {**state, "data": {"response": message}, "long_term_memory": long_term_memory}

```

- # === PERSONALIZED ADVICE ===: Comment indicating the personalized advice logic.
- async def provide_advice(state: FinanceState) -> FinanceState:: Defines an asynchronous function to provide financial advice and returns an updated FinanceState.
- It extracts user_input, user_profile, and long_term_memory from the state.
- It constructs a prompt for the LLM to provide personalized advice based on the user_input and user_profile, also considering any previous_advice given (from long-term memory). It instructs the LLM to use clear, empathetic language suitable for users with limited financial literacy.
- response = await llm.ainvoke(prompt): Calls the LLM to generate the advice.
- message = response.content.strip(): Cleans the LLM's response.
- long_term_memory['last_advice'] = message: Stores the generated advice in the long-term memory.
- It returns a new FinanceState with the data containing the advice message and the updated long_term_memory.

Step 12: Human-in-the-Loop Function

```

1  # === HUMAN-IN-THE-LOOP ===
2  async def human_in_the_loop(state: FinanceState) -> FinanceState:
3      user_input = state['user_input']
4      prompt = (
5          f"The query '{user_input}' has been flagged as high-risk. "
6          f"Judges to a human financial advisor: This query requires review by a financial advisor. "
7          f>Please wait for expert input before proceeding."
8      )
9      message = prompt
10     return {**state, "data": {"response": message}}

```

- # === HUMAN-IN-THE-LOOP ===: Comment indicating the human intervention logic.
- async def human_in_the_loop(state: FinanceState) -> FinanceState:: Defines an asynchronous function that generates a message indicating that human intervention is needed for a high-risk query.
- It extracts the user_input from the state.
- It creates a prompt (which is also used as the message) informing the user that their query is high-risk and requires human review.
- It returns a new FinanceState with the data containing this message.

Step 13: Fallback Function

```

1  # === FALLBACK ===
2  async def fallback(state: FinanceState) -> FinanceState:
3      message = "😞 Sorry, I didn't understand. Try asking about stocks, expenses, budgets, or financial advice."
4      return {**state, "data": {"response": message}}

```

- # === FALLBACK ===: Comment indicating the fallback logic.
- async def fallback(state: FinanceState) -> FinanceState:: Defines an asynchronous function that provides a default response when the user's intent cannot be determined.
- It creates a generic "didn't understand" message.
- It returns a new FinanceState with the data containing this fallback message.

Step 14: Graph Node Selection Function

```

1 # === BUILD GRAPH ===
2 def get_next_node(state: FinanceState) -> str:
3     if state.get("hitl_flag", False):
4         return "human_in_the_loop"
5     valid_intents = ["profile", "stock", "expense", "budget", "advice"]
6     return state["intent"] if state["intent"] in valid_intents else "fallback"

```

- # === BUILD GRAPH ===: Comment indicating the graph building section.
- def get_next_node(state: FinanceState) -> str:: Defines a function that determines the next node in the LangGraph based on the current FinanceState.
- It first checks if the hitl_flag is True. If so, it directs the flow to the "human_in_the_loop" node.
- If the hitl_flag is False, it checks if the detected intent is one of the valid intents. If it is, it returns the intent as the name of the next node.
- If the intent is not valid (e.g., "unknown"), it directs the flow to the "fallback" node.

Step 15: Building the LangGraph

```

1 builder = StateGraph(FinanceState)
2 builder.add_node(INTENT_DETECTION_NODE, detect_intent)
3 builder.add_node("Collect User Data", collect_user_data)
4 builder.add_node("Stock Info", get_stock_info)
5 builder.add_node("Expense Tracker", track_expenses)
6 builder.add_node("Budget Summary", budget_summary)
7 builder.add_node("Provide Advice", provide_advice)
8 builder.add_node("Human in the Loop", human_in_the_loop)
9 builder.add_node("Fallback", fallback)
10 builder.set_entry_point(INTENT_DETECTION_NODE)
11
12 builder.add_conditional_edges(
13     INTENT_DETECTION_NODE,
14     get_next_node,
15     {
16         "profile": "Collect User Data",
17         "stock": "Stock Info",
18         "expense": "Expense Tracker",
19         "budget": "Budget Summary",
20         "advice": "Provide Advice",
21         "human_in_the_loop": "Human in the Loop",
22         "fallback": "Fallback"
23     }
24 )
25 finance_bot = builder.compile()

```

- `builder = StateGraph(FinanceState)`: Creates an instance of the `StateGraph` class, specifying that the state will be managed using the `FinanceState` TypedDict.
- `builder.add_node(...)`: Adds each of the previously defined asynchronous functions as nodes in the graph, with a unique name.
- `builder.set_entry_point(INTENT_DETECTION_NODE)`: Sets the starting node of the graph to the "Intent Detection" node.
- `builder.add_conditional_edges(...)`: Defines the transitions from the `INTENT_DETECTION_NODE`.
 - The first argument is the starting node.
 - The second argument is the `get_next_node` function, which determines the next node based on the current state.
 - The third argument is a dictionary mapping the possible outputs of `get_next_node` (the intents or "human_in_the_loop" or "fallback") to the names of the corresponding nodes.
- `finance_bot = builder.compile()`: Compiles the `LangGraph` into an executable object.

Step 16: Streamlit User Interface



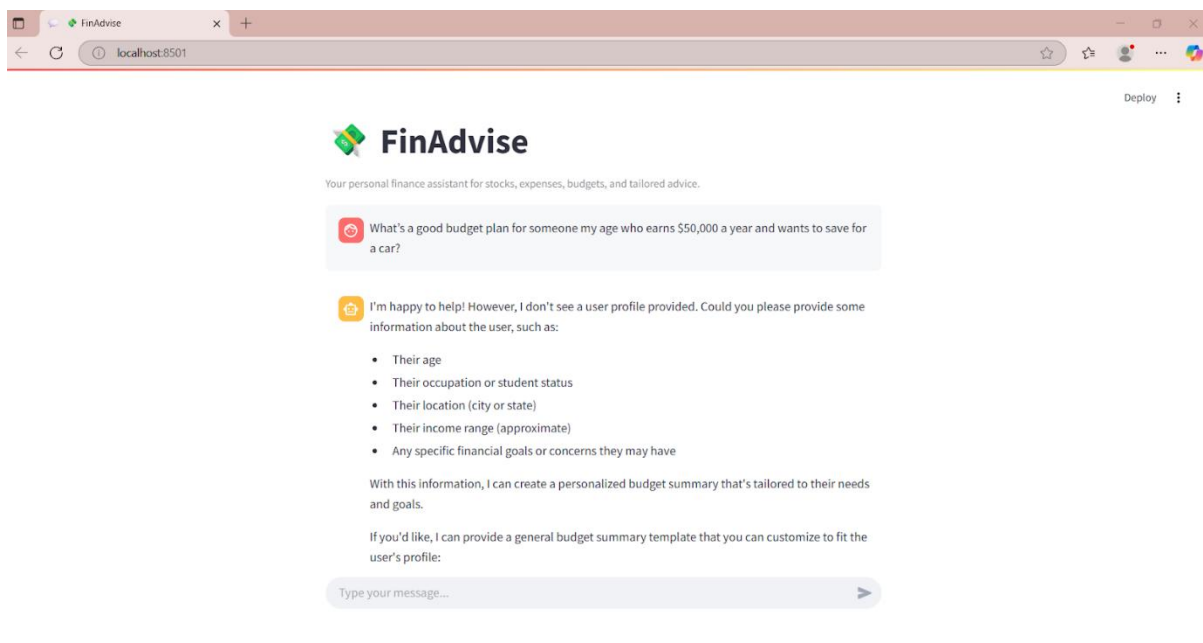
```

1  # === STREAMLIT UI ===
2  st.set_page_config(page_title="🦉 FinAdvise", page_icon="🗨️", layout="centered")
3  st.title("🦉 FinAdvise")
4  st.caption("Your personal finance assistant for stocks, expenses, budgets, and tailored advice.")
5
6  if "messages" not in st.session_state:
7      st.session_state.messages = []
8  if "long_term_memory" not in st.session_state:
9      st.session_state.long_term_memory = {}
10
11 for message in st.session_state.messages:
12     with st.chat_message(message["role"]):
13         st.markdown(message["content"])
14
15 if user_input := st.chat_input("Type your message..."):
16     st.session_state.messages.append({"role": "user", "content": user_input})
17     with st.chat_message("user"):
18         st.markdown(user_input)
19     with st.chat_message("assistant"):
20         with st.spinner("Thinking..."):
21             state = {
22                 "user_input": user_input,
23                 "intent": None,
24                 "data": None,
25                 "user_profile": st.session_state.get("user_profile", {}),
26                 "short_term_memory": {},
27                 "long_term_memory": st.session_state.long_term_memory,
28                 "hitl_flag": False
29             }
30             final_state = asyncio.run(finance_bot.ainvoke(state))
31             bot_reply = final_state['data']['response']
32             st.session_state.user_profile = final_state.get('user_profile', {})
33             st.session_state.long_term_memory = final_state.get('long_term_memory', {})
34             st.markdown(bot_reply)
35     st.session_state.messages.append({"role": "assistant", "content": bot_reply})

```

- `# === STREAMLIT UI ===`: Comment indicating the Streamlit UI section.
- `st.set_page_config(...)`: Configures the title, icon, and layout of the Streamlit page.
- `st.title("🦉 FinAdvise")`: Sets the main title of the application.
- `st.caption(...)`: Adds a smaller caption below the title.
- `if "messages" not in st.session_state: st.session_state.messages = []`: Initializes an empty list in Streamlit's session state to store chat messages if it doesn't exist. This allows the chat history to persist across reruns.
- `if "long_term_memory" not in st.session_state: st.session_state.long_term_memory = {}`: Initializes an empty dictionary for long-term memory in the session state if it doesn't exist.
- `for message in st.session_state.messages::` Loops through the stored messages and displays them in the chat interface using `st.chat_message`.
- `if user_input := st.chat_input("Type your message...")::` Creates a chat input field for the user to type messages. If the user enters a message:

- `st.session_state.messages.append({"role": "user", "content": user_input})`: Appends the user's message to the chat history.
- `with st.chat_message("user"): st.markdown(user_input)`: Displays the user's message in the chat interface with the "user" role.
- `with st.chat_message("assistant")`: Creates a container for the assistant's response.
 - `with st.spinner("Thinking...")`: Displays a spinner animation while the assistant is processing.
 - `state = {...}`: Creates the initial FinanceState dictionary with the user's input and initializes other state components. It retrieves the `user_profile` and `long_term_memory` from the session state to maintain persistence.



FinAdvise

Your personal finance assistant for stocks, expenses, budgets, and tailored advice.



Can you remind me of the budget plan we discussed?

Income: \$4,500 per month

Fixed Expenses (50%): \$2,250

- Rent/Mortgage: \$1,500
- Utilities (water, electricity, gas, internet): \$100
- Groceries: \$300
- Transportation (car payment, insurance, gas): \$250
- Minimum debt payments (credit cards, loans): \$100
- Subscription services (Netflix, gym, software): \$50

Variable Expenses (30%): \$1,350

- Entertainment (dining out, movies, hobbies): \$500
- Travel: \$200
- Miscellaneous (gifts, unexpected expenses): \$150
- Personal care (haircuts, beauty products): \$100
- Pet expenses: \$100
- Home maintenance (repairs, decor): \$100

Savings and Debt Repayment (20%): \$900

- Emergency fund: \$200

Type your message...

